

LogRAG: Semi-Supervised Log-based Anomaly Detection with Retrieval-Augmented Generation

Wanhao Zhang², Qianli Zhang^{*1,2}, Enyu Yu³, Yuxiang Ren^{*3}, Yeqing Meng², Mingxi Qiu², and Jilong Wang^{1,2}

¹Zhongguancun Laboratory, Beijing, China

²Institute for Network Sciences and Cyberspace, Tsinghua University, Beijing, China

³Advance Computing and Storage Lab, Huawei Technologies, China

Email: {zwh22, meng-yq21, choumx23}@mails.tsinghua.edu.cn,

{zhang, wjl}@cernet.edu.cn, {yuenyu, renyuxiang1}@huawei.com

Abstract—Log-based anomaly detection is critical in monitoring the operation of microservice systems and in the real-time reporting of system failures. Utilizing deep learning-based log anomaly detection methods facilitates effective detection of anomalies within logs. However, existing methods are greatly dependent on log parsers, and parsing errors can considerably affect downstream anomaly detection tasks. Additionally, methods that predict the next log event in a sequence are susceptible to the instability of sequences and the emergence of unseen logs as systems evolve, resulting in a higher false positive rate. In this paper, we propose a semi-supervised log anomaly detection framework based on retrieval-augmented generation (RAG). This framework conducts phased detection using both Log Tokens and Log Templates to mitigate the impact of log parsing errors. It also utilizes a single-class classifier to model the normal behavior of the system, thereby circumventing the effects of unstable sequences. Finally, it employs large language model (LLM) empowered by RAG to reevaluate detected anomalous logs.

Index Terms—Log Analysis, Log Anomaly Detection, Deep Learning, Large Language Model, Retrieval-Augmented Generation

I. INTRODUCTION

Microservices offer a more agile and efficient delivery mechanism for web service development. However, as the complexity of functionalities within microservices intensifies, the occurrences of faults and exceptions also increase. These faults not only compromise the user experience but also may precipitate system crashes. As systems grow more intricate, the effectiveness of manual fault detection diminishes. Log files record the runtime states of microservice systems. The prevailing approach to detecting and predicting system anomalies relies on automated log analysis. Therefore, logs are currently widely used in automated monitoring methods for anomaly detection, root cause analysis [1], [2], failure prediction, and other system automation processes. In this paper, our primary focuses on log anomaly detection methods utilizing deep learning.

Log anomaly detection typically involves converting unstructured log information into structured data for analysis,

and the process commonly referred to as log parsing or log template extraction. After acquiring structured log data, deep learning methods are used for system modeling. Most existing log anomaly detection methods are based on predicting the next log event in a sequence and comparing it with actual log events to identify anomalies.

Nevertheless, existing methods face three main challenges. (1) **Log parsing errors**. Current log anomaly detection methods heavily depend on log parsers. However, log parsers may not accurately parse keywords and parameters, leading to the omission of crucial semantic information. These semantic details could be crucial for determining whether a log is anomalous. (2) **Sequence modeling difficulty**. Existing methods typically require sequence modeling; however, unstable sequences can arise during system evolution and updates. Therefore, employing sequence modeling necessitates frequent retraining to update the recorded states of system operations. On the other hand, sequence modeling methods must train models using log sequences of specified window sizes. This precludes flexible adjustment of the granularity of the anomaly detection window. (3) **Unseen logs issue**. As systems evolve, many "unseen logs" may emerge in the training phase. Existing methods often struggle to adapt to these unseen logs, leading to the misclassification of numerous unseen normal logs as anomalies and consequently generating a significant number of false positives.

To address the above challenges, we present LogRAG, a semi-supervised log anomaly detection framework that utilizes RAG for post-processing. Similar to other semi-supervised learning methods, LogRAG only requires training with logs from the system's normal operation state, tolerating a small amount of anomalous logs without the need for manually labeled data. By utilizing Log Token, LogRAG mitigates the risk of log parsing errors, while the deep single-class classifier learns the system's normal behavior without relying on sequence information, thereby mitigating the impact of unstable log sequences. For logs detected as anomalies, we design a post-processing stage based on RAG for reevaluation. Through retrieving log templates from historical normal data, constructing prompts, and utilizing LLM for anomaly detection, LogRAG significantly decreases false positives.

^{*}This research was conducted during Wanhao Zhang's internship at Zhongguancun Laboratory. Qianli Zhang and Yuxiang Ren are the corresponding authors.

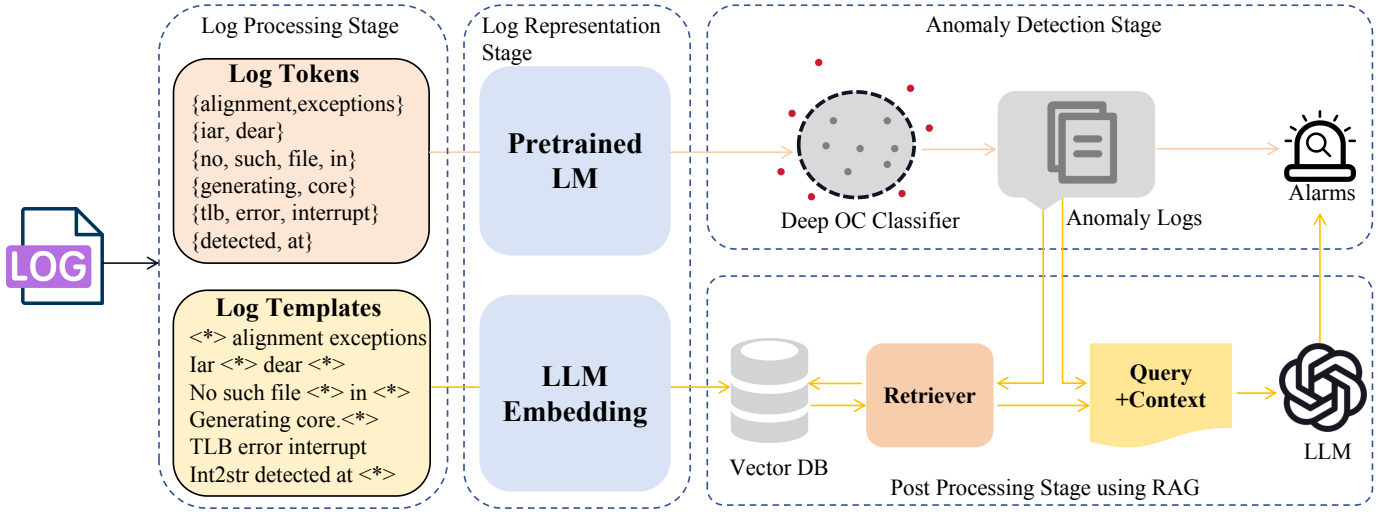


Fig. 1: The overview of LogRAG. It processes logs into Log Tokens and Log Templates. Log Tokens are encoded using pre-trained language models, while Log Templates are encoded using LLM Embedding. **In the training phase**, a deep one-class classifier model is constructed using Log Tokens, and Log Templates are stored in a Vector database. **In the detection phase**, test data is passed through the classifier, and anomalies undergo Post Processing using RAG for final anomaly detection.

II. DESIGN OF LOGRAG

The framework of LogRAG, shown in Fig. 1, consists of four stages, which are briefly described as:

- **Log Processing:** Logs are converted from raw, semi-structured texts to structured data using the log parser. The parsed logs produce Log Tokens and Log Templates. Log Templates abstract logs into common structures, reducing redundancy and storage costs, which optimizes model training and LLM efficiency. Log Tokens are used for model training and inference to mitigate parsing errors. Inspired by NeuralLog [3], we tokenize log content, remove non-character tokens, convert letters to lowercase, and apply camel case splitting to handle composite units.
- **Log Representation:** In the Log Representation phase, we extract semantic representations from Log Tokens and Log Templates using pre-trained models. Pre-trained language models like BERT, while powerful, require fine-tuning on log data for optimal performance. To ensure a comprehensive understanding of log data, we embed log templates using our LLM's embedding method. This alignment maintains semantic consistency, deepens our understanding of queries and contexts, thereby leading to more accurate and insightful analyses.
- **Log Anomaly Detection**
Existing log anomaly detection methods often rely on modeling log sequences, which is impractical due to the unpredictable nature of system-generated logs. Therefore, we adopt a novel approach: modeling the normality of normal logs, without the need for complex log sequence information. We utilize DeepSVDD [4] as the backbone for learning the normality of normal logs in the anomaly detection model training. In the specific context of our task, our objective is to map the features of normal log

templates to the interior of a hypersphere using a neural network. Normal log templates should ideally be close to the center of the hypersphere. To achieve the above goal, the objective function is presented in Eq (1).

$$\min_{R, \mathcal{W}} R^2 + \frac{1}{\nu n} \sum_{i=1}^n \max \left\{ 0, \|\phi(x_i; \mathcal{W}) - \mathbf{c}\|^2 - R^2 \right\} + \frac{\lambda}{2} \sum_{\ell=1}^L \|\mathbf{W}^\ell\|_F^2 \quad (1)$$

Where $\phi(x_i; \mathcal{W})$ represents using a neural network mapping function $\phi(\cdot; \mathcal{W})$ with parameter $\mathcal{W}$ to map each input log template x_i to a hyperspace. The center and radius of the hypersphere are denoted by $\mathbf{c}$ and R , respectively. The regularization term with weight λ is used to prevent overfitting. The core objective is to ensure that the samples lie within the smallest possible hypersphere in the mapped hyperspace.

$$\text{score}(\mathbf{x}) = \|\phi(\mathbf{x}; \mathcal{W}^*) - \mathbf{c}\|^2 - R^2 \quad (2)$$

For anomaly detection, we measure the deviation of logs from the hypersphere's center via Eq (2). Logs inside the hypersphere ($\text{score} \leq 0$) are considered normal, while those outside ($\text{score} > 0$) are deemed anomalous. This scoring mechanism simplifies anomaly determination by focusing solely on spatial distance within the feature hyperspace.

- **Post-processing Using RAG:** After anomaly detection, many normal logs may be misclassified as anomalies due to the limitations of the single-class model. To reduce false positives, we perform a secondary assessment using Retrieval-Augmented Generation (RAG). We store Log

Templates in a vector database and, for each detected anomaly, retrieve the most similar templates using cosine similarity. This information is used to construct a prompt for LLM to make the final anomaly determination.

1) **Retrieve Strategies:** To efficiently retrieve log templates similar to the target log from the vector database, we use cosine similarity as our similarity metric. To ensure relevance and precision in our retrieval process, we set a similarity threshold and specify the number of results retrieved. Specifically, we consider only templates with a cosine similarity score exceeding 0.8. Consequently, we designate the top five templates that meet this criterion as the final retrieval results.

2) **Prompt Construction:** Prompts are designed to guide LLMs through the anomaly evaluation process. Each prompt includes task description, input templates, Chain of Thought(CoT) judgment process, and output format. LLMs assess anomalies based on retrieved templates, following a structured judgment process outlined in the prompt.

3) **LLM:** We use specific prompts to guide LLMs, populating them with input and retrieved templates. OpenAI's ChatGPT and open-source LLMs are utilized for evaluation. Responses from LLMs are evaluated for conformity to the specified format, with ChatGPT used for further parsing if needed.

III. EXPERIMENTS

1) **Datasets:** We conduct our experiments on two widely-used open datasets (BGL, Spirit) from LogHub [5]. Each log in these two datasets was manually labeled as abnormal and normal.

2) **Evaluation Metrics:** We adopt the widely-used Precision, Recall, and F1-score for all experimental evaluations. $Precision = \frac{TP}{TP+FP}$ represents the proportion of logs detected as anomalous by the model that are indeed anomalous. $Recall = \frac{TP}{TP+FN}$ represents the proportion of correctly identified anomalous logs among all logs that are actually anomalous. $F1-Score = \frac{2 \times Precision \times Recall}{Precision + Recall}$ provides a comprehensive evaluation by considering both Precision and Recall. TP stands for True Positive, FP for False Positive, TN for True Negative, and FN for False Negative. It is important to emphasize that some recent works shuffle the dataset before subsequent training. However, this approach leads to the model seeing log data from the future during the training phase (i.e., data leakage), resulting in inflated prediction performance [6]. We adhere to real-world scenarios and do not shuffle the data; instead, we partition the data in chronological order.

3) **Preliminary Experiments:** In this experiment, we preliminarily validated the significant effect of RAG on enhancing the precision of log anomaly detection in online scenarios. We use the BGL dataset, divided into six sequential blocks from 0 to 5. For each block i , we train on its normal data and test on block $i + 1$ ($0 \leq i \leq 4$).

Our results, illustrated in Fig.2(a) and Fig.2(b), show a significant increase in precision and F1-Score across blocks

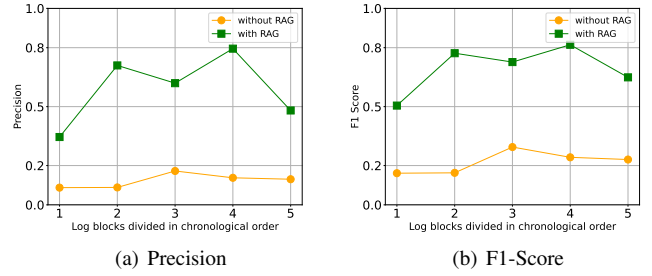


Fig. 2: Precision and F1-Score of LogRAG applying on BGL dataset in chronological order with and without RAG

1 to 5 after integrating RAG. This enhancement in precision consequently improves the F1-Score, indicating that our approach primarily boosts the final anomaly detection performance by increasing the precision of anomaly detection. This improvement reduces manual screening and speeds up decision-making for staff.

IV. CONCLUSION AND FUTURE WORK

This paper presents LogRAG, a semi-supervised framework for log anomaly detection. It incorporates a deep single-class classifier for anomaly detection and employs RAG for post-processing. LogRAG alleviates the effects of log parsing errors and unstable log sequences resulting from system evolution. Moreover, through the utilization of RAG, LogRAG diminishes false positives in the final outcomes. The preliminary experiments have confirmed the effectiveness of LogRAG in log anomaly detection tasks. Our research is ongoing, and in the future studies, we intend to further refine LogRAG's design, compare it with established log anomaly detection methods, and assess the contributions of its individual modules.

V. ACKNOWLEDGMENTS

This work is partially supported by Ascend Servers provided by Advanced Computing and Storage Lab, Huawei.

REFERENCES

- [1] Enyu Yu, Hui Dong, Yuxiang Ren, Minzhi Yan, Xuechang Zhang, Yi Yang, Le Yue, and Zhengbin Huang. Hrca: A heterogeneous graph-based adaptive root cause analysis framework. In *2023 IEEE 34th International Symposium on Software Reliability Engineering Workshops (ISSREW)*, pages 63–68. IEEE, 2023.
- [2] Yeqing Meng, Qianli Zhang, Xiangyu Tang, Wanhao Zhang, and Jilong Wang. Ftm-rca: A fast two-stage multi-dimensional root-cause analysis of network anomalies. In *2023 IEEE/ACM 31st International Symposium on Quality of Service (IWQoS)*, pages 01–10. IEEE, 2023.
- [3] Van-Hoang Le and Hongyu Zhang. Log-based anomaly detection without log parsing. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 492–504. IEEE, 2021.
- [4] Lukas Ruff, Robert Vandermeulen, Nico Goernitz, Lucas Deecke, Shoaib Ahmed Siddiqui, Alexander Binder, Emmanuel Müller, and Marius Kloft. Deep one-class classification. In *International conference on machine learning*, pages 4393–4402. PMLR, 2018.
- [5] Shilin He, Jieming Zhu, Pinjia He, and Michael R Lyu. Loghub: A large collection of system log datasets towards automated log analytics. *arXiv preprint arXiv:2008.06448*, 2020.
- [6] Van-Hoang Le and Hongyu Zhang. Log-based anomaly detection with deep learning: How far are we? In *Proceedings of the 44th international conference on software engineering*, pages 1356–1367, 2022.